

# Modular Point Cloud Viewer with Data Identification and Labeling

Xingying Li; Yufei Hu

February 5, 2024

# 1 Introduction

- **Topic and Purpose:**

This report introduces a significant advancement in the field of construction technology through the development of a Modular Point Cloud Viewer. The purpose of this initiative is to address the inefficiencies in the processing of large-scale point cloud data, a common challenge faced in the construction industry. By leveraging Python's computational capabilities, the project aims to enhance data analysis, interpretation, and utilization for construction projects.

The algorithms implemented within the Python code enable point cloud data noise reduction through spatial filtering using axis-aligned bounding boxes, a common method for isolating regions of interest in point clouds. The data structures, primarily *numpy* arrays, manage the vast data inherent in point cloud processing efficiently, leveraging their capabilities for quick operations on large datasets.

Furthermore, the code integrates GUI actions with point cloud manipulation, allowing for an interactive and user-friendly interface that responds to mouse-click events for data selection and manipulation. Such a system is at the forefront of modern point cloud processing, demonstrating the practical application of algorithms and data structures in a real-world scenario.

- **Document Structure and Content Preview:**

The structure of the document follows a logical progression, beginning with an in-depth analysis of the current challenges associated with point cloud data management. It then transitions into a detailed description of the proposed solution, including the development process, technologies, and methodologies employed. Subsequent sections provide an evaluation of the outcomes, highlighting the features and benefits of the viewer. The final segments discuss the implications for future work and potential integration with other construction management systems.

## 2 State of the Art

- **Current Challenges in Point Cloud Processing:**

The current landscape in point cloud processing is fraught with challenges stemming from the data's volume and complexity. Traditional methods of processing are not only time-intensive but also prone to inaccuracies due to the subjective nature of manual data handling. Automation in this field remains limited, and there's a pressing need for tools that can streamline the workflow without compromising on the data's integrity.

- **Advancements in Data Processing Algorithms:**

In recent years, there has been a considerable shift towards using Python's powerful libraries to process point cloud data. With the rise of open-source platforms like Open3D, developers are now equipped with tools that were once limited to specialized software, and highlights the use of advanced Python algorithms to improve the efficiency and accuracy of processing large point clouds, with a focus on modularity to facilitate easy manipulation and integration into construction applications. These advancements have led to the development of more sophisticated algorithms that can handle large datasets with improved accuracy and efficiency.

- **Transition to Enhanced Visualization Tools:**

Visualization tools have evolved to provide more dynamic and intuitive ways to interact with 3D data. The migration from older systems to modern environments such as PyQt and OpenGL has facilitated the creation of custom widgets that allow for real-time data manipulation and rendering, providing insights that were previously difficult to ascertain. During our build process, the open3d library showed better compatibility and adaptability than PyQt. It describes the transition from PPTK to Open3D for better compatibility with Python 3.6+ and other improvements, like the intuitive PyQt interface for visualization and efficient '.ply' file handling.

- **Application of Algorithms in Point Cloud Segmentation:**

The application employs Open3D for point cloud manipulation, engaging the `segment_plane` method, indicative of a RANSAC algorithm, for plane segmentation. Subsequent denoising is executed via an AxisAligned-BoundingBox filter, which trims the cloud per specified coordinates. Clustering leverages the DBSCAN algorithm (`cluster_dbscan method`), enabling feature differentiation within the point cloud. Users can interactively select and label data points through a PyQt5-based GUI, enhancing the segmentation process. The output—a segmented, denoised, and labeled point cloud—can be exported via the `write_point_cloud` function, illustrating the synergy between automated algorithms and user-guided refinement in point cloud processing. This method segregates the point

cloud data into clusters based on density, allowing for the isolation of different features within the data, such as walls or windows in a building structure.

The classification used color-coding to differentiate between various elements within the segmented point cloud, enabling users to visually distinguish between features. The interactive aspect of the GUI allows users to manually select and categorize points, supplementing the automated segmentation with user input, which can be essential for complex or ambiguous point cloud structures where algorithmic segmentation may need refinement.

The structure of the code suggests a workflow where point cloud data is loaded, segmented into meaningful clusters, interactively labeled, and then saved or further processed. This process underscores the advanced capabilities of Python and associated libraries in handling computationally intensive tasks like point cloud segmentation and classification, which are crucial in fields such as construction.

- **Integration of Open-Source Tools and Hardware in Point Cloud Processing:**

The synergy between open-source tools and advanced hardware has allowed for a more cohesive approach to point cloud processing. By leveraging software and hardware capabilities, researchers and practitioners can capture, process, and analyze data with greater finesse and detail. It integrates tools such as Open3D, PotreeConverter, and CloudCompare, along with hardware components like the *Riegl Scanner* for obtaining and processing point cloud data.

- **Custom Viewer Development for Enhanced Data Interaction:**

Custom Viewer Development for Enhanced Data Interaction and opened new paths for data interaction: A customized point cloud viewer was developed, offering features for intuitive and quick data labeling and display, including a toolbar for modular operations. By utilizing the inherent capabilities of Python and associated libraries, these viewers provide a platform for more nuanced control and manipulation of point cloud data, give understanding of the spatial characteristics of the datasets.

- **Future Outlook - Semi-supervised Labeling and BIM Integration:**

Looking ahead, the integration of semi-supervised machine learning algorithms promises to further automate the process of feature recognition

within point clouds. Furthermore, the potential for integration with Building Information Modeling (BIM) systems paves the way for a more integrated approach to construction and architectural design, signaling a transformative change in how we process and utilize spatial data.

### 3 Problem Statement

Regular batch processing of Point Clouds data in construction applications is obstructed by the time-consuming and laborious due to subjective manual analysis and modeling procedures. Particularly in the context of construction applications, non-manual selection is unfeasible. The limitations associated with the use of cameras or scanners equipped with sRGB sensors, leading to under or overexposed images, further compound the problem. A statement from the EU Construction Foundation underscores the impact of this limitation on data quality: "The limitation of a camera or scanner equipped with an sRGB sensor is often identified as images being under or overexposed, which directly affects the quality of the data"

Particularly, several challenges are encountered when batch processing point cloud data of constructions, mainly due to the sheer volume and complexity of the data. The effective utilization of construction point cloud data in large-scale projects is hindered by a notable requirement of manual data handling, particularly in the context of massive and batch operations. The intricate nature of point cloud datasets, acquired through laser scanning or other surveying methods, presents challenges that can be attributed to the absence of modular approaches.

Despite these challenges, the inherently structured and relatively consistent characteristics of point cloud data in construction scenarios offer a promising avenue for breakthroughs in data acquisition of desired point cloud objects. This insight paves the way for an automated solution aimed at enhancing efficiency and accuracy in the handling of Point Clouds data, with a specific focus on the construction sector.

This project is aimed at solving the high workload in extracting exterior wall and window data from facade point cloud datasets. And to improve the level of classification of individual segments

## 4 Project Goal

The overarching goal of this project is to address the challenge of extracting precise exterior wall and window data from the dataset of facade point clouds. The primary focus is on developing algorithms to accurately identify and delineate exterior walls and windows within the point cloud dataset associated with building facades. In this project, a suite of sophisticated Python algorithms has been deployed to discern and categorize data extracted from point clouds. This emphasis is placed on elevating the efficiency and precision of processing extensive point cloud datasets within the construction domain.

The intrinsic modularity of this algorithmic approach confers a noteworthy advantage, allowing for facile manipulation and seamless integration within diverse construction applications. This modularity ensures adaptability to varying dataset characteristics and analysis requirements, enhancing the versatility of the algorithmic framework.

Consequently, as an integral component of this endeavor, a user-friendly and intuitively designed point cloud viewer is slated for implementation. This viewer is purposefully engineered to align with the principles of modularization, facilitating streamlined operations for users engaging with diverse point cloud datasets. The envisioned viewer aims not only to enhance the visualization of point cloud data but also to provide an intuitive interface for users to interact with modular functionalities, thereby contributing to a more efficient and user-friendly processing environment within the construction context.

Additionally, another key objective is to establish a comprehensive visualization operating platform. This platform will serve as a facilitator for subsequent modular processing of the extracted wall and windows data. The envisioned platform aims to enhance the interoperability and usability of the extracted information, providing a structured environment for efficient and modular analysis of facade elements. Through the integration of advanced visualization techniques and modular processing functionalities, the platform is intended to streamline the exploration and utilization of exterior wall and window data in the context of building facades.

## 5 Concept

- The integration of PPTK-Viewer with PyQt interfaces enables vivid 3D visualization of point cloud data. The code snippet efficiently reads '.ply' files, extracts vertex coordinates and color data, and displays them using the viewer's robust rendering engine. PPTK Viewer's direct rendering from PyQt showcases the potential for seamless, interactive data analysis applications.

However, PPTK's reliance on outdated Python versions limits its compatibility, prompting a shift to alternatives like Open3D that support modern Python environments.

### 5.1 Algorithmic Overview

This subsection gives an overview of the algorithms used in the project.

#### 5.1.1 File Handling

- This application is used to open and save files, evident from the *openProject* and *save* layer functions. It uses *QFileDialog* as a graphical interface to handle file input and output operations. The *mainwindow* class in the provided code exemplifies a sophisticated approach to file management in point cloud visualization applications. The use of *QFileDialog* is particularly noteworthy, as it provides an intuitive interface for users to open and save .ply files, which is crucial for 3D data representation. The *openProject* function of this class is not only a gateway for file access, but also the first step in a pipeline that implements complex data processing. The selection of file formats, filtering options, and integration with native file system dialogue demonstrate a user-centered design approach that prioritizes ease of use without sacrificing functionality. This level of document processing demonstrates that the system was designed with end-user workflows in mind, aiming to simplify the interaction between complex data sets and the professionals who use them.

#### 5.1.2 Point Cloud Processing

- The denoising method implemented within the *mainwindow* class showcases an advanced utilization of spatial data structures, such as the *Ax-alignedBoundingBox*, to refine point cloud data. By establishing a bounding box, the application can effectively isolate sections of the point cloud, enhancing the clarity of the visualized data. This operation is not just a technical procedure but a critical step towards achieving higher data quality. The ability to focus on specific data regions enables users to perform targeted analyses, eliminating extraneous information that may detract from the task at hand. Such a feature is a testament to the application's

robust processing capabilities, reflecting an understanding of the nuanced needs of data-intensive environments.

### 5.1.3 Visualization

- The *OpenGLWidget* class outlines a custom widget for displaying point clouds. It utilizes the *GLViewWidget* from *pyqtgraph.opengl* for rendering and interacting with 3D graphics. In the realm of 3D data visualization, the *OpenGLWidget* class stands out as a cornerstone for rendering point clouds. Its integration with *pyqtgraph.opengl* harnesses the power of *OpenGL* within a Python context, bridging the gap between high-performance graphics programming and accessible scripting. The class's methods provide users with the ability to interact dynamically with the visualization, manipulating the viewpoint with intuitive controls (See Fig.1). This interactive component is not just about viewing data but engaging with it in a manner that promotes a deeper understanding of the spatial relationships and structures within the point cloud. It exemplifies modern visualization techniques, where data is not static but in an immersive and manipulable environment.

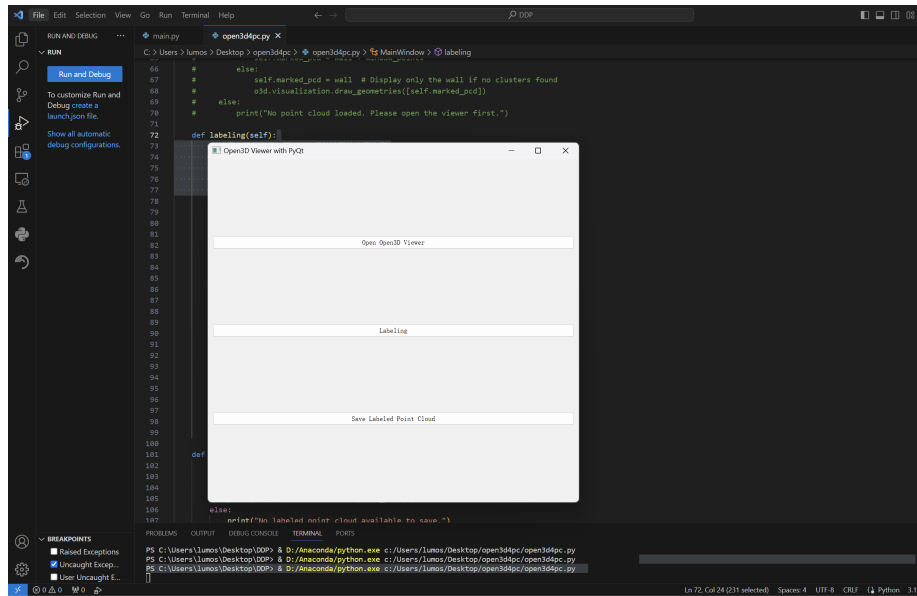


Figure 1: Open3D Labeling Viewer

### 5.1.4 Selection and Labeling

This code allows users to use mouse interaction to click or frame-select (checkbox) parts of the point cloud model and display it in the viewer on the right. The

processed point cloud module can be saved to a folder of your choice depending on the situation.

#### 5.1.5 Data Structures

- The provided Python interface facilitates user interaction with point cloud data, enabling the selection of individual points through mouse events and the application of labels. The labeling function is particularly notable, as it leverages the *RANSAC* algorithm to differentiate between planes within the point cloud, allowing users to classify and segregate data based on geometric criteria. This capacity for labeling directly within the visualization framework not only streamlines the workflow but also enhances the accuracy of data interpretation. Such an approach underscores the application's capability to handle complex data sets, offering tools that cater to the nuanced requirements of spatial analysis in construction and other industries. The emphasis on intuitive design ensures that the selection and labeling processes are both efficient and user-friendly, catering to professionals who may not be well-versed in programming, thereby simplifying access to sophisticated data analysis tools.

### 5.2 Data Structures Usage

- The application's backbone is built on the robust *NumPy* library, which is central to managing and processing the voluminous data involved in point clouds. *NumPy* arrays provide a high-performance container for large data sets, allowing for quick and efficient operations such as indexing, slicing, and broadcasting. This makes them ideal for handling multi-dimensional data in point cloud visualization. By using these arrays, the application can perform complex mathematical computations with relative ease, translating to a more responsive user experience. The data structure's design is inherently optimized for large-scale data manipulation, which is a cornerstone of point cloud analysis, where the ability to handle and process large volumes of data in real-time can significantly impact the outcomes of a project.

### 5.3 Point Cloud Viewer

To perform labeling, editing, and other operations on the processed point cloud data, a desktop point cloud viewer was utilized in this project. This viewer, leveraging the capabilities of Potree, represents a remarkable browser-based tool developed by *Markus Schütz*, significantly transforming the paradigm of interacting with point cloud data. Potree facilitates the seamless visualization and navigation of extensive point cloud datasets, presenting a user-friendly interface endowed with diverse functionalities.

A customized viewer powered by Potree was developed and implemented into this project, aiming to simplify and adapt the operational logic of modularized data, with the addition of new category buttons.

Noteworthy attributes encompass the capacity for distance measurements and point navigation, as well as cross-sectional perspectives. As shown in Fig.2, especially notable features lie in its capability to associate point clouds with attributes and display material colorization styles, thereby augmenting contextual richness and fostering a deeper comprehension of the captured environment.

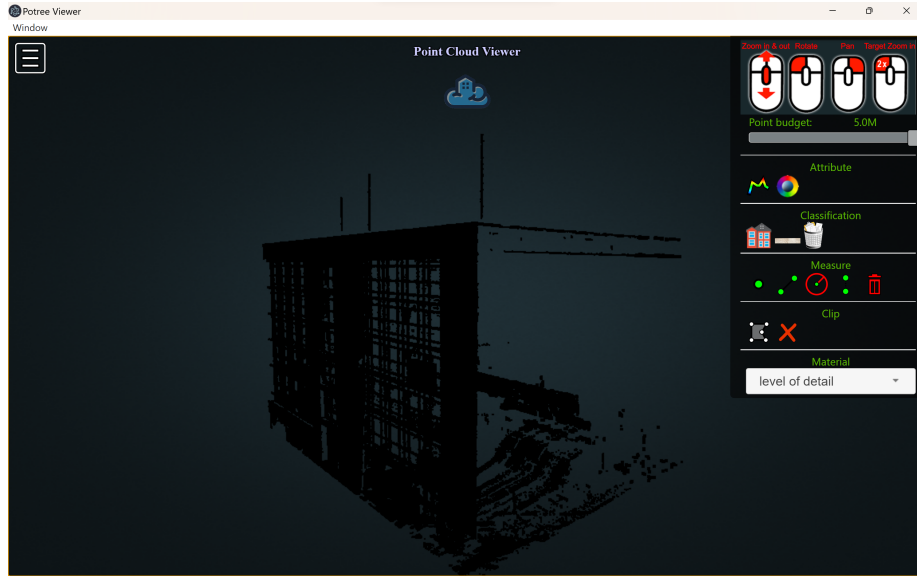


Figure 2: Point Cloud Viewer

## 6 Result

In this project, accurate identification and delineation of facades and windows in the point cloud data set related to building facades is successfully achieved by using Python algorithms.

The Open3D process, integrated with a PyQt snippet, adeptly undertakes the reading of '.ply' files, extracting both vertex coordinates and color data. Subsequently, these data are presented through the viewer's resilient rendering engine. The direct rendering capability of the Open3D Viewer within the PyQt environment exemplifies the ability to create smooth and interactive applications dedicated to data analysis. The result is illustrated in Fig.3 .

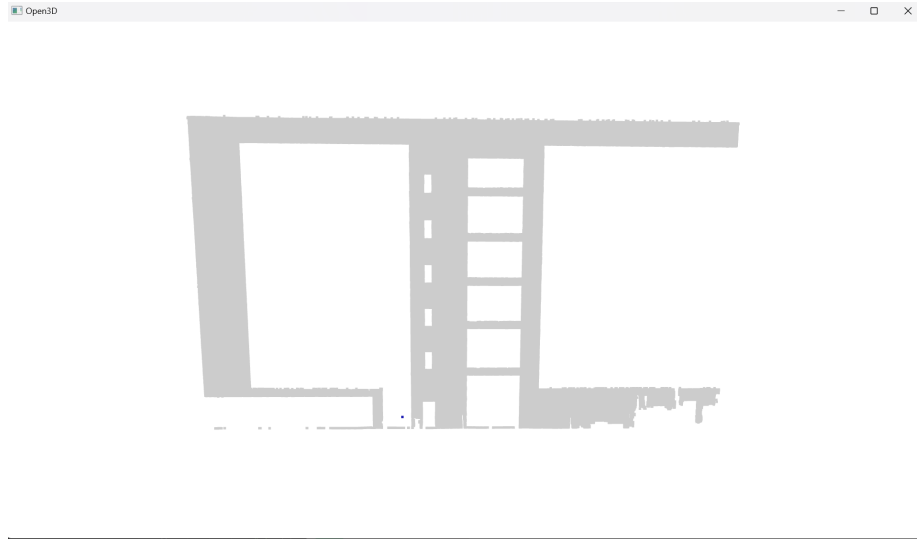


Figure 3: Labeled exterior wall

As mentioned in the preceding section, this project develops a PyQt-based tool for visualizing and selecting point clouds using Open3D. The viewer interface consists of left and right windows, dedicated to editing and viewing point cloud (.ply) data, respectively. The editor window facilitates noise reduction and supports click-select and checkbox-select operations on .ply files. The selected data is rendered in the right display window as an object.

It is noteworthy that, in contrast to conventional solutions, the point-and-click feature relies on characteristic value recognition, enabling its application to irregular and complex shapes, as depicted in Fig. 4: The checkbox-select feature is designed for identifying flat, regular targets by recognizing surface data and successively completing the 3d data into a whole object, as illustrated in Fig.5 .

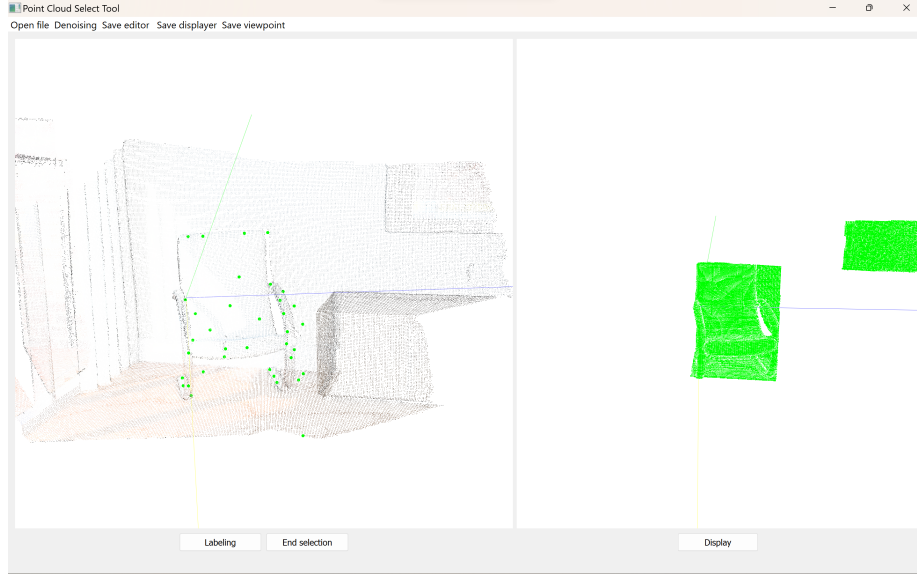


Figure 4: checkbox-select feature

In addition, the objects in the editor and displayer windows can be saved separately for subsequent processing, The Save viewpoint feature supports saving the last selection object for easier categorization of feature values. (Fig.5)

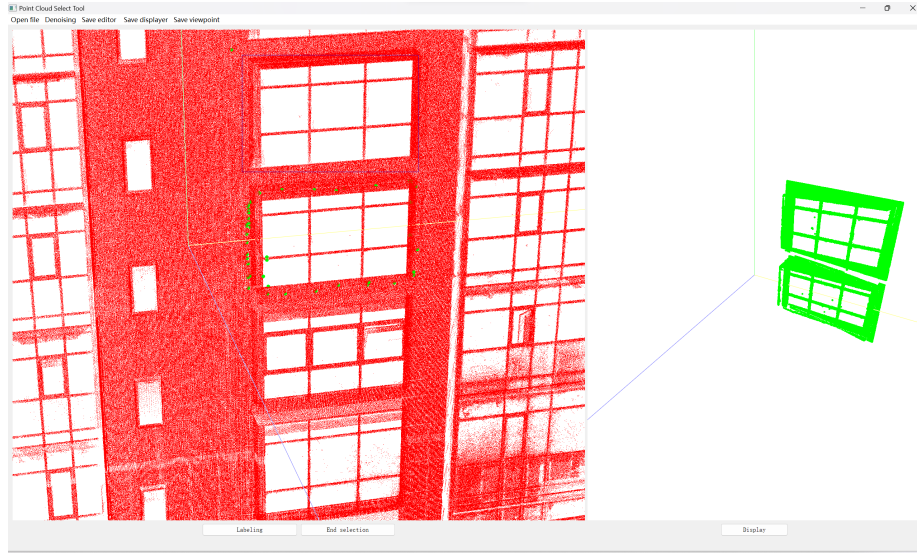


Figure 5: Selection on facade

The post-categorization labeling processed files can be viewed in Point Cloud Viewer after being converted to *Json* format by *PotreeConverter*. This process enables subsequent modularized operations and batch processing and provides expandability for labeling and annotating various elements. Fig.6 illustrates the effect of the exterior wall on the viewer.

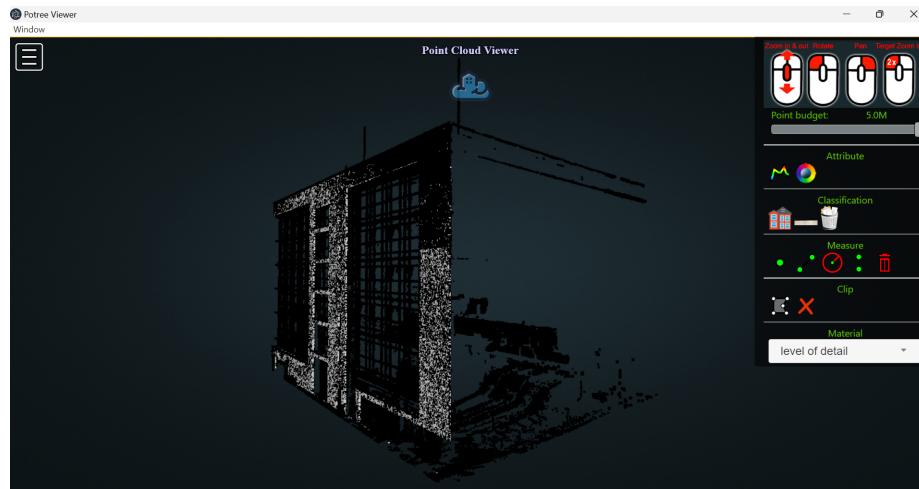


Figure 6: Effect of Exterior Wall on the Viewer

## 7 Problems during the project

The biggest challenge in this project was the processing and optimization of the original .ply point cloud data. As mentioned above, Open3d, as well as the other Python algorithms such as RANSAC, are highly dependent on the quality of the dataset. In this project, the optimization and noise reduction of the point cloud database are based solely on spatial filtering using axis-aligned bounding boxes. As a common and primitive method for isolating regions of interest in point clouds, it may not satisfy every scenario.

As an adventurous attempt, this project proposes to use the RANSAC algorithm along with clustering filtering and labeling of the window structure of the facade dataset. Nevertheless, due to the structure of the original point cloud data as well as the noise reduction strategy, the results are not entirely satisfactory, see Fig. 7.

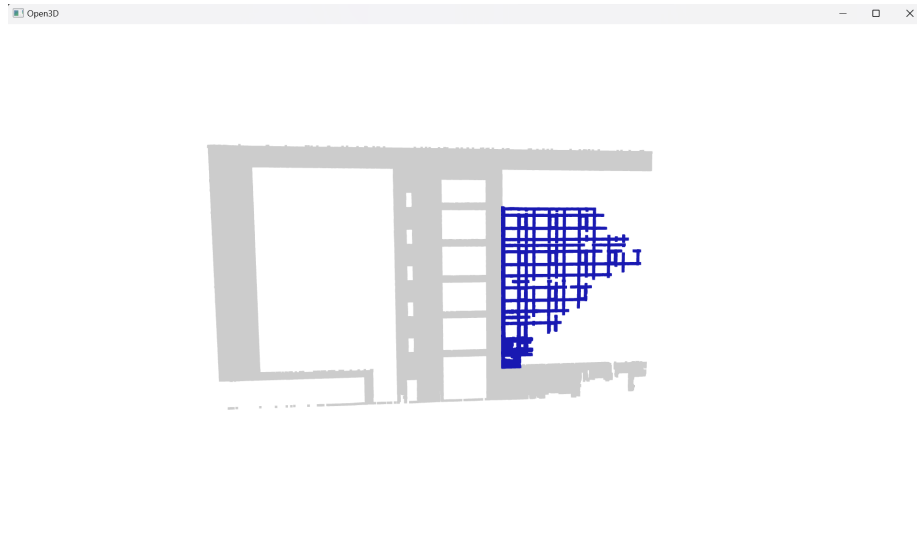
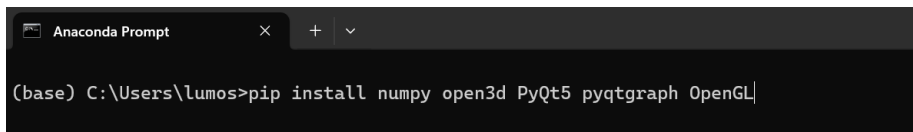


Figure 7: Labeled Wall and Windows Frame

## 8 Instructable-Style Documentation

### 8.1 Setting Up the Environment

- Install a Python Integrated Development Environment (IDE) like PyCharm, Visual Studio Code, or Jupyter Notebook to run Python scripts.
- Ensure Python 3. is installed on your system. You can download it from the official Python website.
- Install necessary libraries using pip:



```
Anaconda Prompt
(base) C:\Users\lumos>pip install numpy open3d PyQt5 pyqtgraph OpenGL
```

Figure 8: Setting environment

### 8.2 Understanding the Main Application (main.py)

- The `main.py` script is the entry point of the application. It initializes the main window and connects UI actions to functions.
- It includes functions to load point cloud files (`openProject`), apply noise reduction (`denoising`), save processed data (`saving_ply`), and visualize the results (`visual_2`).

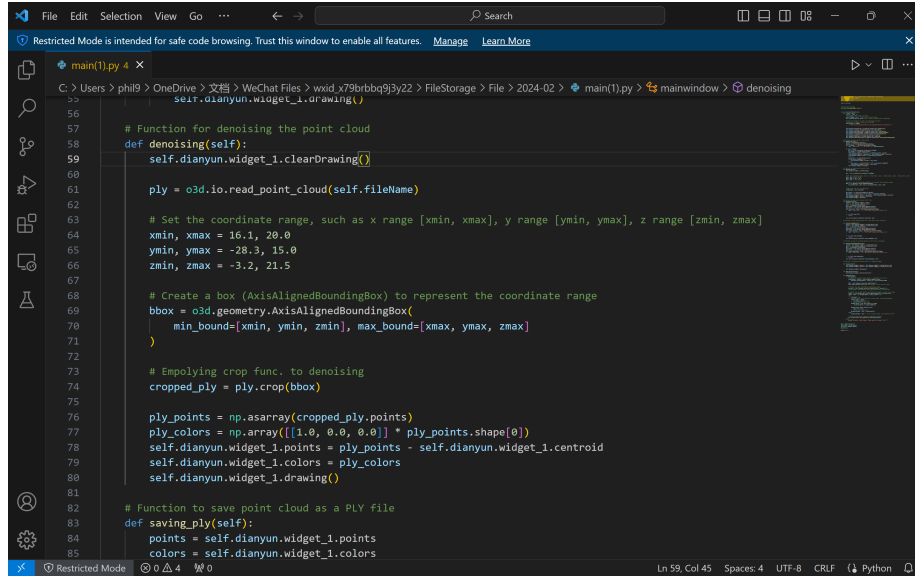


Figure 9: Denoising Function

### 8.3 Visualization Widgets (opengl.py, opengl\_2.py)

The `opengl.py` and `opengl_2.py` scripts define OpenGL widgets for the PyQt5 interface. These widgets are responsible for rendering the point cloud data and handling user interactions like selections and labeling.

The `open3d4pc.py` script likely contains functions for point cloud segmentation using Open3D. The segmentation could involve plane detection or clustering algorithms.

### 8.4 Point Cloud Segmentation (open3d4pc.py)

The `open3d4pc.py` script likely contains functions for point cloud segmentation using Open3D. The segmentation could involve plane detection or clustering algorithms.

```

def labeling(self):
    if self.dianyuns:
        # Segment the largest plane, which is considered as the wall
        plane_model, inliers = self.dianyuns.segment_plane(
            distance_threshold=0.01, ransac_n=3, num_iterations=1000
        )
        wall = self.dianyuns.select_by_index(inliers)
        wall.paint_uniform_color([0.8, 0.8, 0.8]) # Paint the wall gray

        # Extract the rest, potentially containing windows
        non_wall = self.dianyuns.select_by_index(inliers, invert=True)
        # non_window = self.dianyuns.select_by_index(inliers, invert=True)

        # Cluster the non-wall points and assume the smallest cluster is a window
        labels = np.array(non_wall.cluster_dbSCAN(eps=0.02, min_points=10))
        label, count = np.unique(labels, return_counts=True)
        if (
            len(label) > 1
        ): # Ensure there is at least one cluster apart from the background
            window_label = label[
                np.argmax(count[1:]) + 1
            ] # Avoid label -1 for noise
            window_points = non_wall.select_by_index(
                np.where(labels == window_label)[0]
            )
            window_points.paint_uniform_color(
                [0.1, 0.1, 0.7]
            ) # Paint the window blue
            marked_dianyuns = wall + window_points

```

Figure 10: Segmentation

## 8.5 Running the Application

- Open the terminal or command prompt and navigate to the directory containing the scripts.
- Run the main application script:
- The GUI should now be open, allowing you to load and process point cloud data.

## 8.6 Loading and Processing Point Cloud Data

- Use the GUI's file dialog to load a .ply or .pcd file.
- Apply denoising and segmentation algorithms through the provided buttons or menus.
- Use the visualization widget to interact with the point cloud data, selecting regions or points of interest.

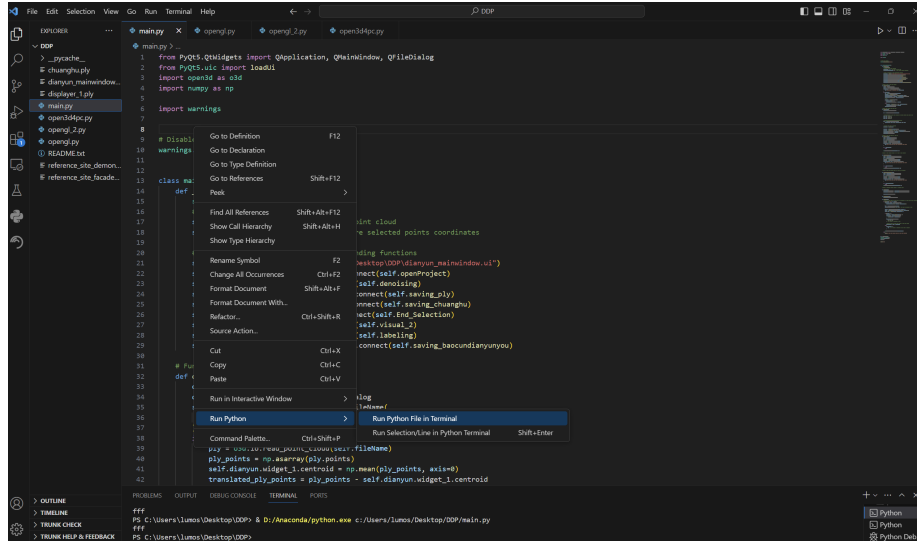


Figure 11: Open main

## 8.7 Exporting the Processed Data

After processing and segmentation, save the modified point cloud using the 'Save' function in the GUI. Select the desired output format and destination folder in the dialog.

## 8.8 Further Steps

- For advanced processing, we can modify the Python scripts to include additional algorithms or customization according to your project needs.
- Experiment with different parameters for segmentation and visualization to achieve optimal results.

## 9 Conclusion and Outlook:

### 9.1 Conclusion

This project successfully implements the initial idea of embedding the concepts of modularity and batch labeling into the operation of the point cloud and implements the fundamental selection and modularity recognition functions through Python algorithms.

Within the domain of construction point cloud data manipulation, this project mitigates the labor associated with data processing, thereby enhancing operational efficiency. In the context of construction applications, notably, the labeling of exterior walls proves instrumental in quantifying paint requirements, evaluating reflectivity, addressing noise control considerations, and other practical applications. Particularly, the selection tool employed in this project stands out for its distinctive point-and-click checkbox-selection features and cataloging processing logic, proving valuable in the realm of construction point cloud data manipulation. The tool's unique attributes, such as point-and-click functionality and systematic cataloging, render it a potent asset for comprehensive construction point cloud analysis. Additionally, the incorporation of a user-friendly and lightweight point cloud viewer further contributes to the comprehensive examination and classification of data within the overarching framework of the project.

### 9.2 Outlook

- Optimize the clustering algorithm to detect point cloud data and enhance the noise reduction strategy so that the labeling and calibration tools can be applied to more construction scenarios.
- **Semi-supervised Labeling:** Deploy Python-based algorithms for the automated annotation of patterns and features in point cloud data. These algorithms are ideally endowed with advanced capabilities in pattern recognition, enabling the precise identification of complex structures and configurations. Additionally, they are designed to facilitate characteristic extraction, allowing for the systematic retrieval of key attributes and defining features inherent in the point cloud data. This comprehensive approach will enhance the efficiency and accuracy of the annotation process, thereby contributing to a more nuanced and detailed analysis of the underlying patterns and features present in the dataset.

- **BIM Integration Capability:** Ensure seamless compatibility with prevalent Building Information Modeling (BIM) software, exemplified by Autodesk Revit, to streamline and enhance the process of exporting markup data. This involves establishing a robust interface that enables the harmonious exchange of annotated information between the developed algorithms and the BIM software, thereby optimizing the system's interoperability. This integration promotes efficiency in the extraction and transfer of pertinent data for further analysis and utilization within the BIM environment.

## References

- Ester, M., Kriegel, H.-P., Sander, J., Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. In Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96) (pp. 226-231). Portland, Oregon: AAAI Press.
- M. A., Fischler, R. C. Bolles, (1981). Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. Communications of the ACM, 24(6), 381-395.
- Makus, Schuetz. <https://github.com/potree/potree>
- Abreu, N., Pinto, A., Matos, A., Pires, M. (2023). Procedural Point Cloud Modelling in Scan-to-BIM and Scan-vs-BIM Applications: A Review. ISPRS International Journal of Geo-Information, 12(7), 260. <https://doi.org/10.3390/ijgi12070260>
- Zeng, R., Shi, J. J. S., Wang, C., Lu, T. (2021). Integrating as-built BIM model from point cloud data in construction projects. Engineering, Construction and Architectural Management. ISSN: 0969-9988.
- Maalek, R. (2021). Field Information Modeling (FIM)<sup>TM</sup>: best practices using point clouds. Remote Sensing, 13(5), 967. <https://doi.org/10.3390/rs13050967>
- Fiawo, J. (2022). Automatically labelling buildings in point clouds using BGT and AHN. Retrieved from <https://openresearch.amsterdam/nl/page/83888/automatically-labelling-buildings-in-point-clouds-using-bgt-and-ahn>